

安全方案

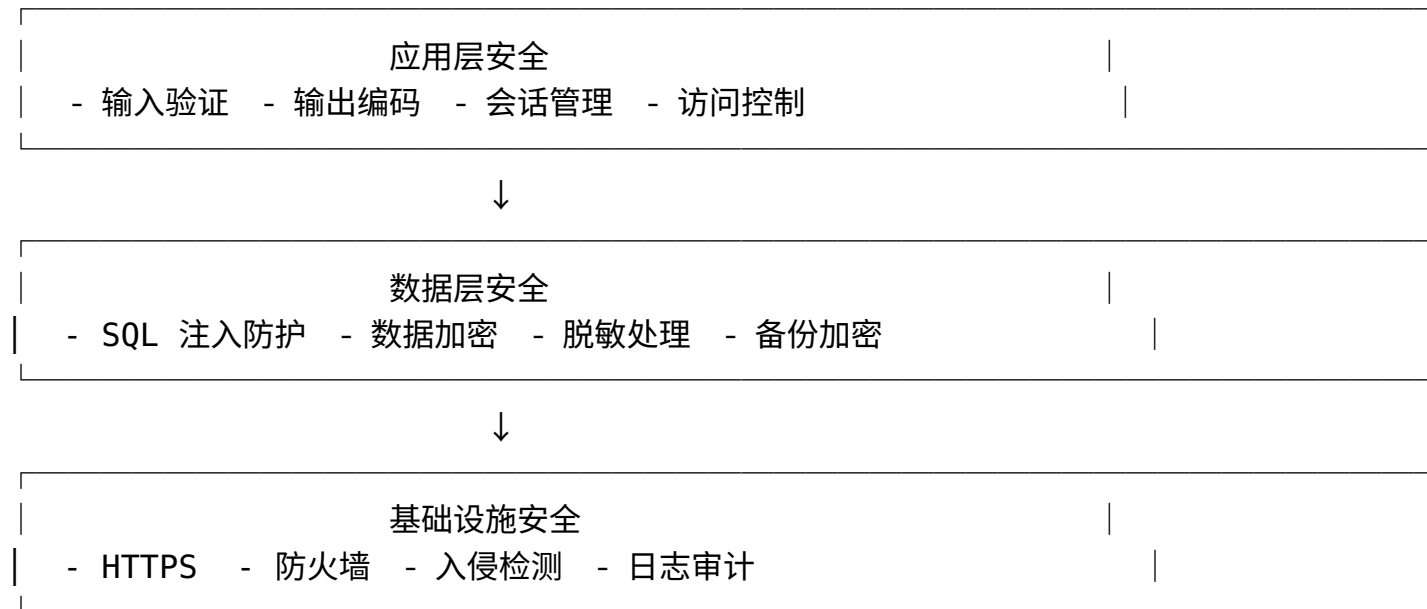
文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、安全架构

1.1 安全分层



1.2 安全原则

原则	说明
最小权限	用户只拥有完成工作所需的最小权限
纵深防御	多层安全防护，不依赖单一措施
默认安全	默认配置为最安全状态
故障安全	系统故障时进入安全状态
完整审计	所有安全相关操作都有日志

二、认证与授权安全

2.1 密码安全

```
// src/utils/password.rs
use argon2::{
    password_hash::{SaltString, PasswordHash, PasswordHasher, PasswordVerifier},
    Argon2, Algorithm, Params, Version
};
use rand_core::OsRng;

/// 密码强度要求
pub struct PasswordRequirements {
    pub min_length: usize,      // 最小长度 8
    pub require_uppercase: bool, // 大写字母
    pub require_lowercase: bool, // 小写字母
    pub require_digit: bool,     // 数字
    pub require_special: bool,   // 特殊字符
}

impl Default for PasswordRequirements {
    fn default() -> Self {
        Self {
            min_length: 8,
            require_uppercase: true,
            require_lowercase: true,
            require_digit: true,
            require_special: false,
        }
    }
}

/// 验证密码强度
pub fn validate_password_strength(password: &str) -> Result<(), PasswordError> {
    let req = PasswordRequirements::default();

    if password.len() < req.min_length {
        return Err(PasswordError::TooShort(req.min_length));
    }
}
```

```

        if req.require_uppercase && !password.chars().any(|c| c.is_uppercase())
            return Err(PasswordError::MissingUppercase);
    }

    if req.require_lowercase && !password.chars().any(|c| c.is_lowercase())
        return Err(PasswordError::MissingLowercase);
    }

    if req.require_digit && !password.chars().any(|c| c.is_numeric()) {
        return Err(PasswordError::MissingDigit);
    }

    // 检查常见弱密码
    let weak_passwords = ["123456", "password", "12345678", "qwerty"];
    if weak_passwords.contains(&password.to_lowercase().as_str()) {
        return Err(PasswordError::TooWeak);
    }

    Ok(())
}

/// 哈希密码
pub fn hash_password(password: &str) -> Result<String, anyhow::Error> {
    let salt = SaltString::generate(&mut OsRng);

    // Argon2id 参数 (OWASP 推荐)
    let params = Params::new(
        19456, // 内存: 19MB
        2,     // 迭代次数
        1,     // 并行度
        Some(Params::DEFAULT_OUTPUT_LEN),
    )?;

    let argon2 = Argon2::new(Algorithm::Argon2id, Version::V0x13, params);

    let hash = argon2.hash_password(password.as_bytes(), &salt)?;
    Ok(hash.to_string())
}

/// 验证密码

```

```
pub fn verify_password(password: &str, hash: &str) -> Result<bool, anyhow::Error> {
    let parsed_hash = PasswordHash::new(hash)?;
    Ok(Argon2::default().verify_password(password.as_bytes(), &parsed_hash))
}
```

2.2 Token 安全

```
// src/utils/token.rs
use jsonwebtoken::{encode, decode, Header, Validation, Algorithm};
use chrono::{Utc, Duration};
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
pub struct Claims {
    pub sub: String, // 用户 ID
    pub username: String,
    pub roles: Vec<String>,
    pub exp: usize, // 过期时间
    pub iat: usize, // 签发时间
    pub jti: String, // Token 唯一标识
}

pub fn generate_access_token(user_id: &str, username: &str, roles: &[String]) -> Result<String, anyhow::Error> {
    let now = Utc::now();
    let exp = now + Duration::minutes(60); // 1 小时有效期

    let claims = Claims {
        sub: user_id.to_string(),
        username: username.to_string(),
        roles: roles.to_vec(),
        exp: exp.timestamp() as usize,
        iat: now.timestamp() as usize,
        jti: Uuid::new_v4().to_string(),
    };

    let secret = std::env::var("JWT_SECRET")
        .expect("JWT_SECRET must be set");

    let token = encode(&Header::default(), &claims, secret.as_bytes())?;
    Ok(token)
}
```

```

}

pub fn verify_token(token: &str) -> Result<Claims> {
    let secret = std::env::var("JWT_SECRET")
        .expect("JWT_SECRET must be set");

    // 验证算法
    let mut validation = Validation::new(Algorithm::HS256);
    validation.validate_exp = true;
    validation.leeway = 60; // 60 秒宽容度

    let token_data = decode::<Claims>(token, secret.as_bytes(), &validation)
        .ok(token_data.claims)
}

// Token 黑名单（用于登出）
pub struct TokenBlacklist {
    blacklist: DashSet<String>,
}

impl TokenBlacklist {
    pub fn add(&self, token: &str, exp: i64) {
        self.blacklist.insert(token.to_string());

        // 设置过期清理
        let token = token.to_string();
        tokio::spawn(async move {
            tokio::time::sleep(Duration::from_secs((exp - Utc::now()).times
            // 清理黑名单
            ));
        });
    }

    pub fn is_blacklisted(&self, token: &str) -> bool {
        self.blacklist.contains(token)
    }
}

```

2.3 防爆破机制

```

// src/middleware/rate_limit.rs
use std::collections::HashMap;

```

```

use tokio::sync::Mutex;
use std::time::{Duration, Instant};

pub struct RateLimiter {
    attempts: Mutex<HashMap<String, Vec<Instant>>>,
}

impl RateLimiter {
    pub fn new() -> Self {
        Self {
            attempts: Mutex::new(HashMap::new()),
        }
    }

    /// 检查是否超过限制
    pub async fn check_rate_limit(
        &self,
        key: &str,
        max_attempts: usize,
        window: Duration,
    ) -> Result<(), RateLimitError> {
        let mut attempts = self.attempts.lock().await;
        let now = Instant::now();

        let user_attempts = attempts.entry(key.to_string()).or_insert_with{

        // 清理过期记录
        user_attempts.retain(|&time| now.duration_since(time) < window);

        // 检查是否超过限制
        if user_attempts.len() >= max_attempts {
            return Err(RateLimitError::TooManyAttempts);
        }

        // 记录本次尝试
        user_attempts.push(now);

        Ok(())
    }
}

```

```
// 使用示例
#[tokio::test]
async fn test_login_rate_limit() {
    let limiter = RateLimiter::new();

    // 前 5 次应该成功
    for _ in 0..5 {
        assert!(limiter.check_rate_limit("user_001", 5, Duration::from_min
    }

    // 第 6 次应该失败
    assert!(limiter.check_rate_limit("user_001", 5, Duration::from_minutes
}
```

三、输入验证与输出编码

3.1 输入验证

```
// src/utils/validation.rs
use validator::{Validate, ValidationError};
use regex::Regex;

/// 案件创建输入验证
#[derive(Debug, Validate, Deserialize)]
pub struct CreateCaseInput {
    #[validate(length(min = 1, max = 200, message = "案件名称长度必须在 1-200
    pub name: String,

    #[validate(length(max = 2000, message = "描述不能超过 2000 字符"))]
    pub description: Option<String>,

    #[validate(length(max = 10, message = "最多 10 个标签"))]
    pub tags: Vec<String>,
}

/// 用户注册输入验证
#[derive(Debug, Validate, Deserialize)]
pub struct RegisterInput {
```

```

#[validate(length(min = 3, max = 50))]
pub username: String,

#[validate(email)]
pub email: String,

#[validate(custom = "validate_password")]
pub password: String,
}

fn validate_password(password: &str) -> Result<(), ValidationError> {
    if password.len() < 8 {
        return Err(ValidationError::new("password_too_short"));
    }

    if !password.chars().any(|c| c.is_uppercase()) {
        return Err(ValidationError::new("password_missing_uppercase"));
    }

    if !password.chars().any(|c| c.is_lowercase()) {
        return Err(ValidationError::new("password_missing_lowercase"));
    }

    if !password.chars().any(|c| c.is_numeric()) {
        return Err(ValidationError::new("password_missing_digit"));
    }

    Ok(())
}

/// 文件名验证
pub fn validate_filename(filename: &str) -> Result<(), ValidationError> {
    // 禁止路径遍历
    if filename.contains("..") || filename.contains("/") || filename.contains("\\") {
        return Err(ValidationError::new("invalid_filename"));
    }

    // 禁止特殊字符
    let invalid_chars = ['<', '>', ':', '"', '|', '?', '*'];
    if filename.chars().any(|c| invalid_chars.contains(&c)) {

```

```

        return Err(ValidationError::new("invalid_filename"));
    }

    // 长度限制
    if filename.len() > 255 {
        return Err(ValidationError::new("filename_too_long"));
    }

    Ok(())
}

```

3.2 SQL 注入防护

// 使用参数化查询 (SQLx 自动防护)

```

// ✅ 正确: 参数化查询
sqlx::query("SELECT * FROM cases WHERE id = ?")
    .bind(&case_id)
    .fetch_one(&pool)
    .await?;

```

```

// ❌ 错误: 字符串拼接 (禁止)
// let query = format!("SELECT * FROM cases WHERE id = '{}'", case_id);

```

3.3 XSS 防护

```

// src/utils/xss.rs
use ammonia::Builder;

```

```

/// HTML 清理
pub fn sanitize_html(input: &str) -> String {
    Builder::new()
        .tags(["b", "i", "em", "strong", "a", "p", "br", "ul", "ol", "li"])
        .strip_comments(true)
        .clean(input)
        .to_string()
}

```

```

/// 输出编码
pub fn html_encode(input: &str) -> String {
    input
}

```

```

        .replace('&', "&amp;")
        .replace('<', "&lt;")
        .replace('>', "&gt;")
        .replace('"', "&quot;")
        .replace('\'', "&#x27;")
    }

// 使用示例
#[test]
fn test_xss_prevention() {
    let malicious_input = "<script>alert('xss')</script>";

    // 清理 HTML
    let sanitized = sanitize_html(malicious_input);
    assert_eq!(sanitized, "alert('xss')");

    // HTML 编码
    let encoded = html_encode(malicious_input);
    assert_eq!(encoded, "&lt;script&gt;alert(&#x27;xss&#x27;)&lt;/script&g");
}

```

四、文件上传安全

4.1 文件类型验证

```

// src/utils/file_security.rs
use mime_guess::Mime;
use std::path::Path;

/// 允许的文件类型
const ALLOWED_MIME_TYPES: [&str] = &[
    "application/pdf",
    "application/msword",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    "application/vnd.ms-excel",
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "image/jpeg",
    "image/png",
    "audio/mpeg",
]

```

```

        "audio/wav",
    ];

    /// 验证文件类型
    pub async fn validate_file_type(
        content: &[u8],
        filename: &str,
    ) -> Result<Mime, FileError> {
        // 1. 检查扩展名
        let ext = Path::new(filename)
            .extension()
            .and_then(|s| s.to_str())
            .ok_or(FileError::InvalidExtension)?;

        let mime = mime_guess::from_ext(ext)
            .first()
            .ok_or(FileError::UnknownMimeType)?;

        // 2. 检查 MIME 类型
        if !ALLOWED_MIME_TYPES.contains(&mime.as_ref()) {
            return Err(FileError::UnsupportedMimeType);
        }

        // 3. 检查文件头 (魔术数字)
        let detected_mime = infer::get(content)
            .map(|m| m.mime_type())
            .unwrap_or("application/octet-stream");

        // 4. 对比声明和检测的 MIME 类型
        if detected_mime != mime.as_ref() && !is_text_file(detected_mime) {
            return Err(FileError::MimeTypeMismatch);
        }

        Ok(mime)
    }

    /// 检查是否为文本文件 (可放宽检查)
    fn is_text_file(mime: &str) -> bool {
        mime.starts_with("text/") || mime == "application/json"
    }

```

4.2 文件上传处理

```
// src/routers/files.rs
use axum::extract::Multipart;
use tokio::fs::File;
use tokio::io::AsyncWriteExt;

pub async fn upload_file(
    State(state): State<Arc<AppState>>,
    claims: Claims,
    Path(case_id): Path<String>,
    mut multipart: Multipart,
) -> Result<Json<EvidenceFile>> {
    // 1. 获取文件字段
    let field = multipart.next_field().await?
        .ok_or(FileError::NoFile)?;

    let filename = field.file_name()
        .ok_or(FileError::MissingFilename)?
        .to_string();

    // 2. 验证文件名
    validate_filename(&filename)?;

    // 3. 读取文件内容
    let data = field.bytes().await?;

    // 4. 验证文件大小
    if data.len() > state.config.max_file_size {
        return Err(FileError::FileTooLarge(state.config.max_file_size));
    }

    // 5. 验证文件类型
    let mime = validate_file_type(&data, &filename).await?;

    // 6. 扫描恶意内容 (可选)
    if let Some(scanner) = &state.virus_scanner {
        if !scanner.scan(&data).await? {
            return Err(FileError::VirusDetected);
        }
    }
}
```

```

}

// 7. 生成安全文件名
let stored_name = generate_safe_filename(&filename);
let storage_path = format!("files/{}/{}", case_id, stored_name);
let full_path = state.config.storage_root.join(&storage_path);

// 8. 确保目录存在
tokio::fs::create_dir_all(full_path.parent().unwrap()).await?;

// 9. 保存文件
let mut file = File::create(&full_path).await?;
file.write_all(&data).await?;

// 10. 创建数据库记录
let evidence = state.db.create_evidence(EvidenceFile {
    id: Uuid::new_v4().to_string(),
    case_id,
    original_name: filename,
    stored_name,
    file_type: mime.to_string(),
    file_size: data.len() as i64,
    storage_path,
    uploaded_by: claims.sub,
    ..Default::default()
}).await?;

Ok(Json(evidence))
}

/// 生成安全的存储文件名
fn generate_safe_filename(original: &str) -> String {
    let uuid = Uuid::new_v4();
    let ext = Path::new(original)
        .extension()
        .and_then(|s| s.to_str())
        .unwrap_or("bin");

    format!("{}", uuid, ext.to_lowercase())
}

```

五、数据加密

5.1 敏感数据加密

```
// src/utils/encryption.rs
use aes_gcm::{
    aead::{Aead, KeyInit, OsRng},
    Aes256Gcm, Nonce
};
use base64::{Engine as _, engine::general_purpose};

/// 加密敏感数据
pub fn encrypt_sensitive_data(data: &str, key: &[u8; 32]) -> Result<String> {
    let cipher = Aes256Gcm::new_from_slice(key)?;
    let nonce = Aes256Gcm::generate_nonce(&mut OsRng);

    let ciphertext = cipher.encrypt(&nonce, data.as_bytes())?;

    // 拼接 nonce 和密文
    let mut encoded = nonce.to_vec();
    encoded.extend_from_slice(&ciphertext);

    Ok(general_purpose::STANDARD.encode(&encoded))
}

/// 解密敏感数据
pub fn decrypt_sensitive_data(encoded: &str, key: &[u8; 32]) -> Result<String> {
    let decoded = general_purpose::STANDARD.decode(encoded)?;

    if decoded.len() < 12 {
        return Err(EncryptionError::InvalidData);
    }

    let nonce = Nonce::from_slice(&decoded[..12]);
    let ciphertext = &decoded[12..];

    let cipher = Aes256Gcm::new_from_slice(key)?;
    let plaintext = cipher.decrypt(nonce, ciphertext)?;
```

```

        Ok(String::from_utf8(plaintext)?))
    }

// 使用示例：加密身份证号
#[tokio::test]
async fn test_encrypt_id_number() {
    let key: [u8; 32] = [0; 32]; // 实际应从环境变量获取
    let id_number = "510101199001011234";

    let encrypted = encrypt_sensitive_data(id_number, &key).unwrap();
    let decrypted = decrypt_sensitive_data(&encrypted, &key).unwrap();

    assert_eq!(decrypted, id_number);
}

```

5.2 数据库加密

```

-- SQLite SQLCipher 加密
-- 在连接字符串中指定密钥
DATABASE_URL=sqlite://./data/legal_minds.db?cipher=aes-256-cbc&key=your-en

-- 或者在代码中设置
PRAGMA key = 'your-encryption-key';
PRAGMA cipher = 'aes-256-cbc';

```

六、日志与审计

6.1 安全日志

```

// src/services/audit_log.rs
use chrono::Utc;
use serde::Serialize;

#[derive(Debug, Serialize)]
pub struct AuditLog {
    pub id: String,
    pub event_type: AuditEventType,
    pub user_id: Option<String>,
    pub ip_address: Option<String>,
    pub user_agent: Option<String>,
}

```

```

    pub resource_type: Option<String>,
    pub resource_id: Option<String>,
    pub action: String,
    pub status: AuditStatus,
    pub details: Option<serde_json::Value>,
    pub created_at: chrono::DateTime<chrono::Utc>,
}

```

```

#[derive(Debug, Serialize, Clone, Copy)]
#[serde(rename_all = "snake_case")]
pub enum AuditEventType {
    Login,
    Logout,
    LoginFailed,
    AccessDenied,
    DataCreate,
    DataUpdate,
    DataDelete,
    PermissionChange,
    ConfigChange,
}

```

```

#[derive(Debug, Serialize, Clone, Copy)]
#[serde(rename_all = "snake_case")]
pub enum AuditStatus {
    Success,
    Failure,
}

```

```

/// 记录安全日志
pub async fn log_audit_event(
    db: &SqlitePool,
    event_type: AuditEventType,
    user_id: Option<String>,
    action: &str,
    status: AuditStatus,
    details: Option<serde_json::Value>,
) -> Result<()> {
    let log = AuditLog {

```

```

        id: Uuid::new_v4().to_string(),
        event_type,
        user_id,
        ip_address: None, // 从请求上下文获取
        user_agent: None,
        resource_type: None,
        resource_id: None,
        action: action.to_string(),
        status,
        details,
        created_at: Utc::now(),
    };

    sqlx::query(
        r#"
        INSERT INTO audit_logs (
            id, event_type, user_id, action, status, details, created_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?)
        "#,
    )
    .bind(&log.id)
    .bind(format!("{:?}", log.event_type))
    .bind(&log.user_id)
    .bind(&log.action)
    .bind(format!("{:?}", log.status))
    .bind(&log.details.map(|d| d.to_string()))
    .bind(&log.created_at)
    .execute(db)
    .await?;

    Ok(())
}

```

6.2 审计日志表

```

CREATE TABLE audit_logs (
    id            TEXT PRIMARY KEY,
    event_type    TEXT NOT NULL,
    user_id       TEXT,
    ip_address    TEXT,

```

```

        user_agent      TEXT,
        resource_type   TEXT,
        resource_id     TEXT,
        action          TEXT NOT NULL,
        status          TEXT NOT NULL,
        details         TEXT,
        created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,

        INDEX idx_event_type ON audit_logs(event_type),
        INDEX idx_user_id ON audit_logs(user_id),
        INDEX idx_created_at ON audit_logs(created_at),
        INDEX idx_status ON audit_logs(status)
    );

-- 安全相关视图
CREATE VIEW security_events AS
SELECT * FROM audit_logs
WHERE event_type IN ('login_failed', 'access_denied', 'permission_change')
ORDER BY created_at DESC
LIMIT 1000;

```

七、安全配置

7.1 环境变量

```

# .env.production
# 认证
JWT_SECRET=your-secure-random-secret-min-32-chars
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7

# 加密
ENCRYPTION_KEY=your-32-byte-encryption-key-here

# 文件上传
MAX_FILE_SIZE=104857600
ALLOWED_FILE_TYPES=pdf,doc,docx,xls,xlsx,jpg,png,mp3,wav

# 安全设置

```

```
CORS_ORIGINS=https://yourdomain.com
CSRF_SECRET=your-csrf-secret
SESSION_SECURE=true
```

```
# 日志
LOG_LEVEL=info
AUDIT_LOG_ENABLED=true
```

7.2 HTTP 安全头

```
// src/middleware/security_headers.rs
use axum::{
    middleware::Next,
    response::Response,
    http::{Request, header},
};

pub async fn security_headers_middleware<B>(
    mut request: Request<B>,
    next: Next<B>,
) -> Response {
    let mut response = next.run(request).await;
    let headers = response.headers_mut();

    // 防止点击劫持
    headers.insert(
        header::X_FRAME_OPTIONS,
        "DENY".parse().unwrap(),
    );

    // 防止 MIME 类型嗅探
    headers.insert(
        header::X_CONTENT_TYPE_OPTIONS,
        "nosniff".parse().unwrap(),
    );

    // XSS 防护
    headers.insert(
        header::X_XSS_PROTECTION,
        "1; mode=block".parse().unwrap(),
    );
}
```

```
);

// 内容安全策略
headers.insert(
    header::CONTENT_SECURITY_POLICY,
    "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-i
    .parse()
    .unwrap(),
);

// 严格传输安全
headers.insert(
    header::STRICT_TRANSPORT_SECURITY,
    "max-age=31536000; includeSubDomains".parse().unwrap(),
);

// Referrer 策略
headers.insert(
    header::REFERRER_POLICY,
    "strict-origin-when-cross-origin".parse().unwrap(),
);

response
}
```

八、安全测试

8.1 依赖安全审计

Rust 依赖审计

```
cargo install cargo-audit
```

```
cargo audit
```

Python 依赖审计

```
pip install safety
```

```
safety check -r python/requirements.txt
```

Node.js 依赖审计

```
npm audit
npm audit fix
```

8.2 OWASP 测试清单

测试项	测试方法	状态
SQL 注入	尝试注入特殊字符	
XSS	尝试注入 <script> 标签	
CSRF	尝试跨站请求	
未授权访问	尝试访问无权限资源	
敏感信息泄露	检查响应头、错误信息	
文件上传漏洞	尝试上传恶意文件	
密码策略	测试弱密码接受	
会话管理	测试 Token 过期	
速率限制	测试爆破防护	

九、总结

9.1 安全措施清单

类别	措施	状态
认证安全	Argon2 密码哈希、JWT Token、防爆破	
授权安全	RBAC、最小权限	
输入验证	参数化查询、HTML 清理、文件类型验证	
数据加密	AES-256 加密、SQLCipher	
日志审计	安全日志、操作日志	
HTTP 安全	安全头、HTTPS	

9.2 安全配置检查

配置项	要求	状态
JWT_SECRET	32+ 字符随机	
HTTPS	强制启用	
CORS	限制源	
文件上传	类型/大小限制	
日志	审计日志启用	

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447